前言

文档简介

介绍RTOS中PWM驱动的接口及使用方法，为PWM使用者提供参考。

目标读者

PWM驱动的开发/维护人员。

适用范围

表1：适用产品列表

产品名称	内核版本	驱动文件
MR527	FreeRTOS	sunxi_pwm_ng.c
AI985	FreeRTOS	sunxi_pwm_ng.c
MR536	FreeRTOS	sunxi_pwm_ng.c
T536	FreeRTOS	sunxi_pwm_ng.c
MR153	FreeRTOS	sunxi_pwm_ng.c
T153	FreeRTOS	sunxi_pwm_ng.c
F135	FreeRTOS	sunxi_pwm_ng.c
F102	FreeRTOS	sunxi_pwm_ng.c

相关术语介绍

术语	解释说明
频率	PWM的频率决定了所模拟电平的平滑度（逼真度）， 人耳感知的频率范围为20Hz-16 kHz，注意PWM的频率不要落在这个区间。
占空比	决定了一个周期内PWM信号高低的比例，进而决定了一个周期内的平均电压，也就是所模拟的电平的电压。
极性	决定了是高占空比的信号输出电平高，还是低占空比信号输出电平高。 假设一个信号的占空比为100%，如果为正常极性，则输出电平最大，如果为翻转的极性，则输出电平为0。
开关	控制PWM信号是否输出。
PWM对	电机等硬件需要两路脉冲信号来控制其正常运转，一般两路极性相关，频率，占空比参数相同的PWM构成一个PWM对。

深圳易新泰科技有限公司YiXinTai

模块介绍

模块功能介绍

脉冲宽度调制（PWM）是一种对模拟信号电平进行数字编码的方法。 通过高分辨率计数器的使用，方波的占空比被调制用来对一个具体模拟信号的电平进行编码。PWM具有以下规格特性：

- 支持脉冲（脉冲个数可配）、周期和互补对输出；
- 支持捕捉输入；
- 带可编程死区发生器，死区时间可控；
- 0-24 MHz/100 MHz输出频率范围、0%-100%占空比可调、最小分辨率1/65536（即1/2^16）；
- 支持PWM输出和捕捉输入产生中断；
- 支持PWM组模式，同组内各个通道起始相位可配置。

模块配置介绍

设备配置说明

PWM模块寄存器的基本配置位于sdk/lichee/rtos-hal/hal/source/pwm_ng/platform/pwm_sun55iw3.h。

```
/* pwm 各个资源的基地址 */
#define  PWM0_CPUX_BASE      0x02000c00UL
#define  PWM1_CPUX_BASE      0x02051000UL
#define  PWM_CPUS_BASE       0x07020c00UL
#define  PWM_DSP_BASE        0x07103000UL

#define PWM_CHAN_NUM         16      /* 一套资源支持的最多通道数目 */
#define PWM_PORT_NUM         4       /* pwm支持资源的最大套数 */
/* CPUX PWM0 config */
#define SUNXI_CPUX_PWM0_CLK_TYPE     HAL_SUNXI_CCU
#define SUNXI_CPUX_PWM0_CLK_BUS_ID   0
#define SUNXI_CPUX_PWM0_CLK_ID       CLK_PWM
#define SUNXI_CPUX_PWM0_RESET_TYPE   HAL_SUNXI_RESET
#define SUNXI_CPUX_PWM0_RESET_ID     RST_BUS_PWM
#define PWM0_CPUX_BASE               0x02000c00UL
#define SUNXI_CPUX_IRQ_PWM0          MAKE_IRQn(92, 4)

/* CPUX PWM1 config */
#define SUNXI_CPUX_PWM1_CLK_TYPE     HAL_SUNXI_CCU
#define SUNXI_CPUX_PWM1_CLK_ID       CLK_PWM1
#define SUNXI_CPUX_PWM1_CLK_BUS_ID   0
#define SUNXI_CPUX_PWM1_RESET_TYPE   HAL_SUNXI_RESET
#define SUNXI_CPUX_PWM1_RESET_ID     RST_BUS_PWM1
#define PWM1_CPUX_BASE               0x02051000UL
#define SUNXI_CPUX_IRQ_PWM1          MAKE_IRQn(107, 7)

/* CPUS PWM config */
#define SUNXI_CPUS_PWM_CLK_TYPE     HAL_SUNXI_R_CCU
#define SUNXI_CPUS_PWM_CLK_BUS_ID   CLK_BUS_R_PWM
#define SUNXI_CPUS_PWM_CLK_ID       CLK_R_PWM
#define SUNXI_CPUS_PWM_RESET_TYPE   HAL_SUNXI_R_RESET
```

深圳易新泰科技有限公司YiXinTai

```
#define SUNXI_CPUS_PWM_RESET_ID     RST_R_PWM
#define SUNXI_CPUS_IRQ_PWM          MAKE_IRQn(72, 0)
#define PWM_CPUS_BASE               0x07020c00UL

/* DSP PWM config */
#define SUNXI_DSP_PWM_CLK_TYPE      HAL_SUNXI_DSP
#define SUNXI_DSP_PWM_CLK_BUS_ID    CLK_BUS_DSP_PWM
#define SUNXI_DSP_PWM_CLK_ID        CLK_DSP_PWM
#define SUNXI_DSP_PWM_RESET_TYPE    HAL_SUNXI_DSP_RESET
#define SUNXI_DSP_PWM_RESET_ID      RST_BUS_DSP_PWM
#define SUNXI_DSP_IRQ_PWM           MAKE_IRQn(17, 0)
#define PWM_DSP_BASE                0x07103000UL

/* 不同通道的资源整合 */
#define CPUX_PWM0_PARAMS \
{   .port = 0, \
    .reg_base = PWM0_CPUX_BASE, .irq_num = SUNXI_CPUX_IRQ_PWM0, .gpio_num = 16, \
    .clk_type = SUNXI_CPUX_PWM0_CLK_TYPE, .bus_clk_id =
SUNXI_CPUX_PWM0_CLK_BUS_ID, \
    .clk_id = SUNXI_CPUX_PWM0_CLK_ID, .reset_type = SUNXI_CPUX_PWM0_RESET_TYPE, \
    .reset_id = SUNXI_CPUX_PWM0_RESET_ID, \
}

#define CPUX_PWM1_PARAMS \
{   .port = 1, \
    .reg_base = PWM1_CPUX_BASE, .irq_num = SUNXI_CPUX_IRQ_PWM1, .gpio_num = 4, \
    .clk_type = SUNXI_CPUX_PWM1_CLK_TYPE, .bus_clk_id =
SUNXI_CPUX_PWM1_CLK_BUS_ID, \
    .clk_id = SUNXI_CPUX_PWM1_CLK_ID, .reset_type = SUNXI_CPUX_PWM1_RESET_TYPE, \
    .reset_id = SUNXI_CPUX_PWM1_RESET_ID, \
}

#define CPUS_PWM_PARAMS \
{   .port = 2, \
    .reg_base = PWM_CPUS_BASE, .irq_num = SUNXI_CPUS_IRQ_PWM, .gpio_num = 2, \
    .clk_type = SUNXI_CPUS_PWM_CLK_TYPE, .bus_clk_id = SUNXI_CPUS_PWM_CLK_BUS_ID,
\
    .clk_id = SUNXI_CPUS_PWM_CLK_ID, .reset_type = SUNXI_CPUS_PWM_RESET_TYPE, \
    .reset_id = SUNXI_CPUS_PWM_RESET_ID, \
}

#define DSP_PWM_PARAMS \
{   .port = 3, \
    .reg_base = PWM_DSP_BASE, .irq_num = SUNXI_DSP_IRQ_PWM, .gpio_num = 8, \
    .clk_type = SUNXI_DSP_PWM_CLK_TYPE, .bus_clk_id = SUNXI_DSP_PWM_CLK_BUS_ID, \
    .clk_id = SUNXI_DSP_PWM_CLK_ID, .reset_type = SUNXI_DSP_PWM_RESET_TYPE, \
    .reset_id = SUNXI_DSP_PWM_RESET_ID, \
}

/*****************************************************************************
 * define  gpio
 *****************************************************************************/
typedef struct pwm_gpio_t
{
    gpio_pin_t pwm_pin;
    int pwm_function;
```

```
    bool bind_mode;
    u32 bind_channel;
    u32 dead_time;
} pwm_gpio_t;

/* 各个资源及对应的引脚配置 */
static pwm_gpio_t pwm_gpio[PWM_PORT_NUM][PWM_CHAN_NUM] =
{
    {
        /* CPUX0 PWM0 */
        {
            /* pwm0 */
            .pwm_pin = GPIOD(0),
            .pwm_function = 5,
        },
    },

    {
        /* CPUX1 PWM1 */
        {
            /* pwm0 */
            .pwm_pin = GPIOD(16),
            .pwm_function = 5,
        },
    },

    {
        /* CPUS PWM */
        {
             /* pwm0 */
            .pwm_pin = GPIOL(10),
            .pwm_function = 2,
        },
    },

    {
        /* DSP PWM */
        {
            /* pwm0 */
            .pwm_pin = GPIOL(2),
            .pwm_function = 4,
        },
    }
};
```

kernel menuconfig 配置说明

```
Drivers Options  --->
 soc related device drivers  --->
    SUNXI PWM Devices
        [*]enable pwm-ng driver
        [*]enable pwm-ng hal APIs test command
```

深圳易新泰科技有限公司YiXinTai

模块源码结构

PWM模块源码结构如下所示：

```
hal/source/pwm_ng/ ---- 驱动源码
├── hal_pwm_ng.c
├── Kconfig
├── Makefile
├── platform
|   └── pwm_sun8iw22.h
|   └── pwm_sun55iw3.h
|   └── pwm_sun55iw6.h
└── platform_pwm_ng.h

include/hal/ ---- 驱动APIs声明头文件
└── sunxi_hal_pwm_ng.h

hal/test/pwm_ng/ ---- 驱动APIs测试代码
├── Makefile
└── test_pwm_ng.c
```

模块接口说明

需要包含头文件:

```
#include <sunxi_hal_pwm_ng.h>
```

接口列表

PWM提供的接口列表如下：

```
pwm_status_t hal_pwm_init(hal_pwm_port_t port)
pwm_status_t hal_pwm_control(hal_pwm_port_t port, int channel, struct pwm_config
*config_pwm)
pwm_status_t hal_pwm_control_single(hal_pwm_port_t port, int channel, struct
pwm_config *config_pwm)
pwm_status_t hal_pwm_control_dual(hal_pwm_port_t port, int channel, struct
pwm_config *config_pwm)
```

接口使用说明

不同平台pwm资源介绍

产品名称	pwm所属处理单元	port编号	pwm通道数量
MR527/AI985	CPUX	0	16
MR527/AI985	CPUX	1	16
MR527/AI985	CPUS	2	2
MR527/AI985	DSP	3	8

深圳易新泰科技有限公司YiXinTai

产品名称	pwm所属处理单元	port编号	pwm通道数量
MR536/T536	CPUX	0	10
MR536/T536	CPUX	1	10
MR536/T536	CPUX	2	10
MR536/T536	CPUS	3	4
MR153/T153	CPUX	0	8
MR153/T153	CPUX	1	8
MR153/T153	CPUX	2	10
F135	CPUX	0	8

PWM初始化接口

- 函数原型：pwm_status_t hal_pwm_init(hal_pwm_port_t port)
- 功能描述：程序运行的入口函数。
- 参数说明：port,指使用哪套资源（CPUX、CPUS、DSP）。
- 返回值：=0,运行成功；<0 ,失败。

PWM控制接口

- 函数原型：pwm_status_t hal_pwm_control(hal_pwm_port_t port, int channel, struct pwm_config *config_pwm)
- 功能描述：完成对pwm通道的控制。
- 参数说明：port,指使用哪套资源（CPUX、CPUS、DSP），channel,指向所属的pwm通道；config_pwm，pwm_config类型的指针,包含周期、极性、占空比。
- 返回值：=0,运行成功；<0 ,失败。

单通道控制接口

- 函数原型：pwm_status_t hal_pwm_control_single(hal_pwm_port_t port, int channel, struct pwm_config *config_pwm)
- 功能描述：PWM多通道控制函数。
- 参数说明：port,指使用哪套资源（CPUX、CPUS、DSP），channel,指向所属的pwm通道；config_pwm，pwm_config类型的指针,包含周期、极性、占空比。
- 返回值：=0,运行成功；<0 ,失败。

双通道控制接口

- 函数原型：pwm_status_t hal_pwm_control_dual(hal_pwm_port_t port, int channel, struct pwm_config *config_pwm)
- 功能描述：PWM多通道控制函数。
- 参数说明：port,指使用哪套资源（CPUX、CPUS、DSP），channel,指向所属的pwm通道；config_pwm，pwm_config类型的指针,包含周期、极性、占空比。

深圳易新泰科技有限公司YiXinTai

- 返回值：=0,运行成功；<0 ,失败。

模块使用范例

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <unistd.h>

#include <hal_log.h>
#include <hal_cmd.h>
#include <sunxi_hal_pwm_ng.h>

#if defined(CONFIG_ARCH_DSP)
#include <delay.h>
#define sleep(sec) msleep(sec * 1000)
#endif

#ifndef printfFromISR
#define printfFromISR printf
#endif

static int cmd_test_pwm(int argc, char **argv)
{
    struct pwm_config *config;
    uint8_t port;
    uint8_t channel;
    int period, duty;

    if (argc < 5) {
        hal_log_info("Usage: port | pwm channel | duty | period\n");
        return -1;
    }

    hal_log_info("Run pwm hal layer test case\n");

    // 解析命令行参数：PWM资源编号
    port = strtol(argv[1], NULL, 0);
    hal_log_info("port = %d", port);
    // 解析命令行参数：PWM通道号
    channel = strtol(argv[2], NULL, 0);
    hal_log_info("channel = %d", channel);
    // 解析命令行参数：占空比时间（纳秒）
    duty = strtoul(argv[3], NULL, 0);
    // 解析命令行参数：周期时间（纳秒）
    period = strtoul(argv[4], NULL, 0);

    // 为PWM配置结构体分配内存
    config = (struct pwm_config *)malloc(sizeof(struct pwm_config));

    // 设置PWM配置参数
    config->duty_ns   = duty;         // 设置占空比时间（纳秒）
    config->period_ns = period;     // 设置周期时间（纳秒）
    config->polarity  = PWM_POLARITY_NORMAL;  // 设置极性为正常极性
```

深圳易新泰科技有限公司YiXinTai

```
    hal_log_info("duty_ns = %d \n", config->duty_ns);
    hal_log_info("period_ns = %d \n", config->period_ns);
    hal_log_info("polarity = %d \n", config->polarity);

    // 初始化指定PWM资源
    hal_pwm_init(port);

    // 控制指定PWM通道输出配置的波形
    hal_pwm_control(port, channel, config);

    hal_log_info("control pwm test finish\n");

    return 0;
}

FINSH_FUNCTION_EXPORT_CMD(cmd_test_pwm, hal_pwm, pwm hal APIs tests)

static int cmd_release_pwm_channel(int argc, char **argv)
{
    struct pwm_config *config;
    uint8_t port;
    uint8_t channel;
    uint8_t ns;
    ulong period, duty;

    if (argc < 3) {
        hal_log_info("Usage: pwm port\n");
        return -1;
    }

    hal_log_info("Run close pwm channel test case\n");

    // 解析命令行参数：PWM资源编号
    port = strtol(argv[1], NULL, 0);
    // 解析命令行参数：PWM通道号
    channel = strtol(argv[2], NULL, 0);
    hal_log_info("channel = %d", channel);

    // 释放指定PWM资源中的指定通道
    hal_pwm_release(port, channel);

    hal_log_info("release pwm channel finish\n");

    return 0;
}

FINSH_FUNCTION_EXPORT_CMD(cmd_release_pwm_channel, hal_pwm_close, release pwm
channel hal APIs tests)

static void pwm_cap_callback(void* param)
{
    // PWM捕获回调函数，用于处理捕获到的PWM信号信息
    hal_pwm_cap_info *info = (hal_pwm_cap_info *)param;

    // 打印捕获到的PWM信号参数：通道号、计数值、周期、占空比
```

深圳易新泰科技有限公司YiXinTai

```
    printfFromISR("pwm%d capture callback, cnt is %d, period is %d, duty is
%d\n", info->channel, info->cnt, info->period, info->duty);
}

hal_pwm_status_t pwm_capture_init(uint8_t port, uint32_t channel)
{
    // 初始化PWM捕获功能，启用指定端口和通道的捕获
    // port: PWM资源编号，channel: PWM通道号
    // pwm_cap_callback: 捕获回调函数，当捕获到PWM信号时调用
    hal_pwm_cap_enable(port, channel, pwm_cap_callback);

    return HAL_PWM_STATUS_OK;
}

hal_pwm_status_t pwm_capture_deinit(uint8_t port, uint32_t channel)
{
    // 停用PWM捕获功能，释放指定端口和通道的捕获资源
    // port: PWM资源编号，channel: PWM通道号
    hal_pwm_cap_disable(port, channel);

    return HAL_PWM_STATUS_OK;
}

void cmd_pwm_capture_help(void)
{
    // 打印PWM捕获测试的帮助信息
    printf("pwm capture test, stop capture after 10s\n");
    printf("usage: pwm_capture_test<channel>\n");
    printf("\t<channel>: 0 ~ 15\n");
    printf("eg: pwm_capture_test 0, pwm0 capture function\n");
}

int cmd_pwm_capture(int argc, char *argv[])
{
    int count = 0;
    uint8_t port;
    uint8_t channel;

    // 解析命令行参数：PWM资源编号
    port = strtol(argv[1], NULL, 0);
    // 解析命令行参数：PWM通道号
    channel = strtol(argv[2], NULL, 0);
    if (argc != 3 || channel > 15) {
        cmd_pwm_capture_help();
        return -1;
    }

    /* capture setting */
    // 初始化PWM捕获功能
    pwm_capture_init(port, channel);

    // 等待10秒，期间持续捕获PWM信号
    for(count = 0; count < 10; count++) {
        sleep(1);  // 每次休眠1秒
    }
```

深圳易新泰科技有限公司YiXinTai

```
    /* disable */
    // 停用PWM捕获功能，释放资源
    pwm_capture_deinit(port, channel);

    printf("[%s]: pwm stop capture ssd\n", __func__);

    return 0;
}

FINSH_FUNCTION_EXPORT_CMD(cmd_pwm_capture, pwm_capture_test, pwm capture test)
```

测试

基本功能测试

注意：运行`hal_pwm 0 0 500 1000`，其中，`0 0 500 1000`对应`[PWM资源][PWM通道] [脉冲] [周期]`。

执行`hal_pwm 0 0 500 1000`的关键log和波形图如下：

```
cpu0>hal_pwm 0 0 500 1000
Run pwm hal layer test case

port = 0
channel = 0
duty_ns = 500

period_ns = 1000

polarity = 1

[pwm0] not support in sys_config

control pwm test finish
```

深圳易新泰科技有限公司YiXinTai



PWM对插入死区测试

死区的配置如下所示：

```
352                 {
353                         /* pwm2 */
354                         .pwm_pin = GPIOL(4),
355                         .pwm_function = 4,
356                         .bind_mode = 1,
357                         .bind_channel = 3,
358                         .dead_time = 20,
359                 },
360                 {
361                         /* pwm3 */
362                         .pwm_pin = GPIOL(5),
363                         .pwm_function = 4,
364                 },
365                 {
366                         /* pwm4 */
367                         .pwm_pin = GPIOL(6),
368                         .pwm_function = 3,
369                         .bind_mode = 1,
370                         .bind_channel = 5,
371                         .dead_time = 20,
372                 },
373                 {
374                         /* pwm5 */
375                         .pwm_pin = GPIOL(7),
376                         .pwm_function = 3,
377                 },
378                 {
379                         /* pwm6 */
```

注意：运行`hal_pwm 4 2 8000 20000`，其中，`4 2 8000 20000`对应`[PWM资源][PWM通道] [脉冲(ns)] [周期(ns)]`。

参数计算逻辑说明：

- 脉冲宽度 = 8000 ns = 8 μs

- 周期时间 = 20000 ns = 20 μs
- 占空比 = (脉冲宽度/周期时间) × 100% = (8000/20000) × 100% = 40%
- 频率 = 1/周期时间 = 1/20 μs = 50 kHz

执行`hal_pwm 4 2 8000 20000`波形图如下：



捕获功能测试

注意：运行`pwm_capture_test 0 0`，其中，`0 0`对应`[PWM资源][PWM通道]`。

执行`hal_pwm 0 0`的关键log如下：

```
cpu0>pwm_capture_test 0 0
[pwm0] not support in sys_config

[cmd_pwm_capture]: pwm stop capture ssd

cpu0>
```

捕获计数功能测试

表2：功能适用产品列表

产品名称	内核版本	驱动文件
MR536	FreeRTOS	sunxi_pwm_ng.c
T536	FreeRTOS	sunxi_pwm_ng.c
MR153	FreeRTOS	sunxi_pwm_ng.c
F135	FreeRTOS	sunxi_pwm_ng.c

+----------------+------------------+----------------------+

以pwm0-0作为输出，pwm1-0作为捕获为例，将pwm0-0和pwm1-0引脚短接。

1、pwm0-0输出1KHZ，执行：

```
hal_pwm 0 0 500000 1000000 0 0 0
```

2、pwm1-0捕获1秒脉冲个数，执行：

```
pwm_capture_count_test 1 0 1
```

> usgae: pwm_capture_count_test <timeout_s>

关键log如下：

```
cpu0>hal_pwm 0 0 500000 1000000 0 0 0
Run pwm hal layer test case in cycle mode!

duty_ns = 500000

period_ns = 1000000

polarity = 0

pulse_num = 0

output_mode = 0

control pwm test finish


cpu0>pwm_capture_count_test 1 0 1
[cmd_pwm_capture_count]: pulse_num is: 1000
```

FAQ

当IO口未输出波形或者输出的波形与设置不匹配

可做如下排查：

①排查PWM的IO状态

通过在RV端输入如下命令，将输出io的复用功能dump出来，对比是否一致，检查引脚的配置，例如通过查询《**MR527_User_Manual**》得知PD2的功能引脚寄存器为0x02000090，打印的寄存器值如下：

```
cpu0>p 0x02000090 4
start_addr=0x02000090 end_addr=0x02000094
reg_addr[0x02000090]=0xfffff5f5
reg_addr[0x02000094]=0xffffffff
```

关注第8到11bit为PD2的功能引脚参数，并可参照2.2.2.1的PD2功能引脚寄存器表，第8到11bit的值为5对应PWM功能。

②排查PWM的寄存器状态

dump PWM的功能设置寄存器PCR和周期占空比寄存器PPR，确认设置的参数是否写入到寄存器中，寄存器的相关介绍如下：

深圳易新泰科技有限公司YiXinTai

Offset: 0x0100+N*0x0020			Register Name: PCR
Bit	Read/Write	Default/Hex	Description
7:0	R/W	0x0	PWM_PRESCAL_K PWM pre-scale K, actual pre-scale is (K+1) K = 0, actual pre-scale: 1 K = 1, actual pre-scale: 2 K = 2, actual pre-scale: 3 K = 3, actual pre-scale: 4 ..... K = 255, actual pre-scale: 256

Offset: 0x0100+N*0x0020			Register Name: PCR
Bit	Read/Write	Default/Hex	Description
31:16	R/W	0x0	PWM_PUL_NUM In pulse mode, PWM output pulse for PWM_CYCLE_NUM+1 times and then stop.
15:12	/	/	/
11	R	0x0	PWM_PERIOD_RDY PWM period register ready 0: PWM period register is ready to write 1: PWM period register is busy.
10	R/WAC	0x0	PWM_PUL_START PWM pulse output start 0: no effect 1: output pulse for PWM_CYCLE_NUM+1 After finishing configuration for outputting pulse, set this bit once and then PWM would output waveform. After the waveform is finished, the bit will be cleared automatically.
9	R/W	0x0	PWM_MODE PWM output mode select 0: cycle mode 1: pulse mode
8	R/W	0x0	PWM_ACT_STA PWM active state 0: Low Level 1: High Level

关注PCR寄存器以下bit位：

- 第0~7bit位为cycle的缩放比例，用于计算PPR寄存器的实际值。
- 第8bit位为极性配置，0表示低电平为active状态，1表示高电平为active状态。
- 第9bit位为模式配置，0表示周期模式输出，1表示脉冲模式输出。

Offset: 0x0104+N*0x0020			Register Name: PPR
Bit	Read/Write	Default/Hex	Description
31:16	R/W	0x0	PWM_ENTIRE_CYCLE Number of the entire cycles in the PWM clock 0 = 1 cycle 1 = 2 cycles ..... N = N+1 cycles If the register need to be modified dynamically, the PCLK should be faster than the PWM CLK .
15:0	R/W	0x0	PWM_ACT_CYCLE Number of the active cycles in the PWM clock 0 = 0 cycle 1 = 1 cycles ..... N = N cycles

关注PPR寄存器以下bit位：

- 第0到15bit位为active状态所占的周期，以**cycles的数量**表示对应周期时间。
- 第16到31bit位为整个周期，以**cycles的数量**表示对应周期时间。

以上面PWM0_2输出为例，dump对应的0x2000d40 PCR寄存器和0x2000d44 PPR寄存器，可使用命令

```
p 0x02000d40 4
```

dump寄存器指令如下：

```
cpu0>p 0x2000d40 4
start_addr=0x02000d40 end_addr=0x02000d44
reg_addr[0x02000d40]=0x00000103
reg_addr[0x02000d44]=0xea5f7530
```

其中PPR寄存器中cycle的数量和设置周期时间的计算公式如下：

cycle number = （周期时间 x 时钟频率）/ (PRESCAL x 1000000000)

周期时间为5ms，时钟频率为24M，PRESCAL 为寄存器PCR第0 ~ 7bit值，寄存器值3对应值为4；

可计算active的cycle的数量对应 = （5000000 * 24000000）/ (4 * 1000000000) = 30000；对比active的寄存器cycle 0x7530，配置与计算值一致，整个周期的cycle数量计算一样。

::: note

公式的相关计算过程介绍如下：

由于设置的周期时间是以ns为单位，时钟频率是1/s，在计算对应的cycle数量时，需要进行单位换算除于10的9次方，由于cycle数量值在寄存器里是有上限的，因此引入cycle的倍频PRESCAL ，增大cycle数量的上限。

其中周期时间为自定义的时间，单位是ns，时钟频率为PWM控制器所使用的频率，默认为24M时钟，当设置的周期小于41.6ns（1/24M）时，改用100M时钟（驱动会根据周期时间自动切换选择时钟，无需手动设置）；时钟的选择可dump PCCR寄存器（0x02000c24）对比，具体介绍详见《**MR527_User_Manual**》，这里不再赘述。

从PPR寄存器可看出，数据为16位（最大值为65535）；cycle的数量对应周期时间，假如cycle与频率时间无倍增关系，则65535对应最大时间为65535*（1/24M）与等于2.72ms，明显不能满足我们之前10ms为周期的输出，因此引入cycle的倍增机制，增大每个cycle对应的时间，倍增系数为PCR 0 ~ 7bit位值。以之前10ms周期为例，2.72ms需倍增4倍可达10ms，因此PCR的0 ~ 7bit理论计算为3（寄存器值为3实际增倍为3+1=4）；从寄存器可得出PCR的倍增寄存器为3，符合理论计算（驱动根据周期时间进行倍增值计算，无需手动设置）。

:::

③排查PWM的资源冲突状态

关注Linux端的资源使用，检测该io是否在linux端被使用（可检测linux端最终生成的.sunxi.dts是否有io使用冲突路径在指令cout（out/mr527/evb/openwrt）下），linux是否也有使用到rtos端适配的PWM资源，避免两系统同时使用同一个io或PWM，否则两系统相互影响。

深圳易新泰科技有限公司YiXinTai